

贪心专题（提高班）

刘洋

2026

Table of Contents

- 1 贪心基础与正确性证明
- 2 排序贪心
- 3 区间贪心
- 4 优先队列（堆）贪心
- 5 反悔贪心

Table of Contents

- 1 贪心基础与正确性证明
- 2 排序贪心
- 3 区间贪心
- 4 优先队列（堆）贪心
- 5 反悔贪心

贪心基础与正确性证明

- **贪心思想**：每一步都做出最优的局部选择，并期望这些局部最优也出现在全局最优中。不能被说明最优的贪心策略 ** 尽量不要使用 **。
- **贪心成立的两个支柱**：贪心选择性质（存在一个全局最优解包含当前的贪心选择）+ 最优子结构（做出该选择后，剩余子问题的最优解与原问题最优解可拼接）。
- **交换论证 (exchange argument)**：设最优解 O 与贪心解 G 在某处首次不同，构造性地把 O 中的一对元素交换，证明结果不会更差，从而可逐步把 O 调整成 G 。这是证明**排序贪心**最常用的武器。
- **反证 / 微扰**：假设贪心非最优，找到一个更优解，推出与贪心选择矛盾。
- **常见可贪心的信号**：可按某个键排序后线性处理；选择之间相互独立或有明显的"代价单调"；问题可化为拟阵 / 调度 / 区间结构。
- **警惕**：不能贪心的问题（如 0/1 背包），贪心会给出错误答案，需用 DP。判断标准始终是**能否证明**，而非"看起来对"。

Table of Contents

- 1 贪心基础与正确性证明
- 2 排序贪心
- 3 区间贪心
- 4 优先队列（堆）贪心
- 5 反悔贪心

排序贪心

- **核心套路**：定义一个**比较关系**对元素排序，再线性扫描即可得最优。难点在于"如何排序"以及证明该排序最优。
- **推导排序键的方法**：考察任意**相邻两元素** i, j ，写出" i 在前"与" j 在前"的代价/收益表达式，令 i 在前更优，化简得到一个只关于两元素的不等式，它就是排序的比较函数。
- 例：两元素 i, j 若满足 $A_i B_j < A_j B_i$ 时 i 应在前，则按 A_i / B_i 升序排序（经典如国王游戏、皇后游戏一类的雏形）。
- **为何相邻交换就够**：若排序满足"任意相邻逆序交换都不变差"，由冒泡排序的视角，任意排列都能通过相邻交换变到该序，故该序全局最优——这正是交换论证。
- **注意**：推导出的比较关系必须满足**全序性（传递性）**，否则排序无意义；常见反例需特判。

- 有 n 项工作, 第 i 项耗时 A_i , 必须在截止时刻 B_i (含) 前完成; 一次只能做一项, 做完立刻接着做下一项。问能否全部按时完成。
- 数据范围: $1 \leq n \leq 2 \times 10^5$, $1 \leq A_i, B_i \leq 10^9$ 。

Table of Contents

- 1 贪心基础与正确性证明
- 2 排序贪心
- 3 区间贪心**
- 4 优先队列（堆）贪心
- 5 反悔贪心

区间贪心

- **选最多不相交区间 / 活动选择**: 把区间按右端点升序排序, 从左到右扫, 只要当前区间左端点不与已选区间冲突就选它。" 右端点最小" 留给后面最大的空间, 是这类问题的黄金法则。
- **证明 (交换论证)**: 第一个被选的应是右端点最小者; 若最优解第一个区间右端点更大, 把它替换成右端点最小者不会减少可选数量, 归纳即得。
- **区间覆盖 (用最少数目区间盖住一段)**: 按左端点排序, 在所有左端点 $\leq$ 当前覆盖位置的区间里贪心取右端点最大者。
- **"打点" 对偶**: 用最少的点刺穿所有区间——同样按右端点排序, 每次在最靠右处放点。许多看似图论/链上的删边问题 (如断桥使两点不连通) 都能转成" 区间选点"。
- **带截止的调度**: 常配合优先队列做反悔, 见后两节。

P4053 [JSOI2007] 建筑抢修

- n 个建筑, 修第 i 个耗时 $T_{1,i}$, 且必须在时刻 $T_{2,i}$ 前**修完**才有效 (可瞬移到任意建筑, 但同时只能修一个)。求最多能修好多少个建筑。
- 数据范围: $1 \leq n < 1.5 \times 10^5$, $1 \leq T_{1,i} < T_{2,i} < 2^{31}$ 。

Table of Contents

- 1 贪心基础与正确性证明
- 2 排序贪心
- 3 区间贪心
- 4 优先队列（堆）贪心**
- 5 反悔贪心

优先队列（堆）贪心

- **核心：**贪心每步要"取出当前最优/最劣的元素"，但元素集合在动态变化（边扫边加入、边删除）。用**堆**维护这个集合，把每步的取极值与插入做到 $O(\log n)$ 。
- **典型范式一——按时间/截止扫描：**把任务按时间排序，扫描时把"此刻可用"的任务推入堆，再从堆里取价值最大者执行（赚钱调度、会议安排）。
- **典型范式二——合并代价（Huffman）：**每次取最小的两个合并，把合并结果放回堆；合并 k 叉时注意补虚点使 $(n - 1) \bmod (k - 1) = 0$ 。
- **典型范式三——倒序/差分扫描：**把"释放资源"事件也压进堆，用一个堆装"空闲资源"决定能否安排新任务。
- **要点：**想清楚堆里存什么（剩余收益？耗时？截止？）、按什么比较，以及何时入堆/出堆。堆贪心常与反悔贪心只有一线之隔。

- 有 n 份一次性工作，做第 i 份后要第 A_i 天才拿到报酬 B_i ；每天最多做一份。问从今天起 m 天内（含今天开工）能拿到的报酬最大总和。
- 数据范围： $1 \leq n, m \leq 10^5$, $1 \leq A_i \leq 10^5$, $1 \leq B_i \leq 10^4$ 。

P2949 [USACO09OPEN] Work Scheduling G

- n 项工作, 每项耗时 1 单位; 第 i 项有截止时间 D_i , 在该时刻前完成可得利润 P_i 。每个时间单位只能做一项。求最大总利润。
- 数据范围: $1 \leq n \leq 10^5$, $1 \leq D_i \leq 10^9$, $1 \leq P_i \leq 10^9$ 。

Table of Contents

- 1 贪心基础与正确性证明
- 2 排序贪心
- 3 区间贪心
- 4 优先队列（堆）贪心
- 5 反悔贪心**

反悔贪心

- **动机**：普通贪心一旦做出选择就不能改。当"先按局部最优选，后面发现更好却没空间"时，需要**允许撤销之前的选择**——这就是反悔贪心。
- **反悔自动机（堆撤销）**：边贪心边把"已做的选择"塞进堆。遇到新元素，若直接选会超约束，就比较"放弃堆中代价最大/收益最小的旧选择 + 选新元素"是否更优，是则弹出旧选择完成替换。前面 Work Scheduling、建筑抢修都是此型。
- **反悔边（带悔机会的差分）**：把"撤销某次选择"建模成一个虚拟元素压回堆——选了某物后，往堆里放入一个"反悔项"，其代价为"若放弃它改选他者所需补偿"。Cow Coupons 即典型。
- **形式**：常见两类——(1) 维护"当前已选集合"的堆，替换最差者；(2) 把每次选择产生的"反悔代价"作为新元素入堆，下次可能取出它。
- **口诀**：先大胆地按贪心选，再用堆为"后悔"留一条路。

- 一条直线上有 n 栋楼，相邻间距已知。要选 k 对楼配对（每栋至多属于一对），且配对的两栋必须**相邻选取**，使所选 k 段距离之和最小。
- 数据范围： $n \leq 10^5$ 。